Comparación empírica de los algoritmos de ordenamiento por inserción y Quicksort en C++: tiempo, operaciones y estabilidad

Empirical comparison of insertion sort and Quicksort in C++: running time, operations and stability

[Mauricio Alex Felix Camacho]
Universidad Nacional Jorge Basadre Grohmann, Facultad de Ingeniería
Escuela Profesional de Ingeniería en Informática y Sistemas, Tacna, Perú
ORCID: [0009-0004-1921-5770] · Correo: mfelixc@unjbg.edu.pe

Resumen

Objetivo: comparar empíricamente el ordenamiento por inserción y el quicksort (partición de Lomuto con pivote central) según tiempo de ejecución, comparaciones, intercambios, profundidad de recursión, memoria auxiliar y estabilidad.

Métodos: se implementaron ambos algoritmos en C++ y se ejecutaron sobre cinco patrones de entrada (aleatorio, ascendente, descendente, casi ordenado y con duplicados), con n = 1 000, 10 000 y 100 000 y 30 repeticiones por condición con semillas fijas; se registraron tiempo, comparaciones, intercambios y profundidad de recursión, y se calculó estadística descriptiva.

Resultados: inserción creció de forma cuadrática (pendiente log-log ≈ 2.00), salvo en entradas ascendentes (≈ 0.91), donde fue 13 veces más rápida que quicksort (0.0858 ms frente a 1.10 ms con n = 100 000). Quicksort creció cerca de n log n y ordenó 100 000 enteros aleatorios en 4.29 ms frente a 1 645 ms de inserción, pero con 15 valores únicos subió a 221 ms, con unos 6 807 niveles de recursión. Inserción fue estable y quicksort no.

Conclusiones: el patrón de entrada condiciona la elección del algoritmo: inserción solo conviene con datos ya ordenados o muy pequeños, y esta versión de quicksort necesita un tratamiento específico de los valores repetidos.

Palabras clave: ordenamiento; inserción; quicksort; complejidad algorítmica; estabilidad; C++

Abstract

Objective: to empirically compare insertion sort and quicksort (Lomuto partition with middle pivot) in terms of running time, comparisons, swaps, recursion depth, auxiliary memory and stability.

Methods: both algorithms were implemented in C++ and run on five input patterns (random, ascending, descending, nearly sorted and with duplicates), with n = 1,000, 10,000 and 100,000 and 30 repetitions per condition using fixed seeds; time, comparisons, swaps and recursion depth were recorded and descriptive statistics were computed.

Results: insertion sort grew quadratically (log-log slope ≈ 2.00), except on ascending inputs (≈ 0.91), where it was 13 times faster than quicksort (0.0858 ms vs. 1.10 ms for n = 100,000). Quicksort grew close to n log n and sorted 100,000 random integers in 4.29 ms vs. 1 645 ms for insertion sort, but with 15 distinct values it rose to 221 ms, with about 6 807 recursion levels. Insertion sort was stable and quicksort was not.

Conclusions: the input pattern determines the best choice: insertion sort is only advisable for already sorted or very small data, and this quicksort version needs specific handling of repeated values.

Keywords: sorting; insertion sort; quicksort; algorithmic complexity; stability; C++

1. Introducción

Ordenar una colección de datos es una de las operaciones más frecuentes en el software y una de las más estudiadas en las ciencias de la computación. Más allá de ser un ejercicio clásico, el ordenamiento es un paso previo de la búsqueda binaria, la eliminación de duplicados y muchos procesos de análisis de datos (Cormen et al., 2022). Por ello, elegir bien el algoritmo y conocer su comportamiento real, y no solo el teórico, tiene consecuencias prácticas en el tiempo de respuesta de una aplicación.

El ordenamiento por inserción es un método directo, sencillo y estable cuya complejidad es cuadrática en el caso promedio y en el peor caso, pero lineal cuando la entrada ya está ordenada (Knuth, 1998). El quicksort, propuesto por Hoare (1962), pertenece a la familia de los métodos logarítmicos: su tiempo esperado es del orden de n log n, aunque su peor caso es cuadrático si los pivotes son sistemáticamente malos. Para reducir ese riesgo se emplean variantes como elegir el pivote al azar, en el centro del tramo o por mediana de tres (Sedgewick, 1978). Estas diferencias teóricas se perciben de manera distinta según el tamaño y el orden inicial de los datos, por lo que conviene contrastarlas con mediciones.

Trabajos previos han comparado algoritmos de ordenamiento mediante análisis estadísticos de sus tiempos de ejecución (Salgado et al., 2022) y han evaluado la eficiencia de implementaciones recursivas e iterativas (Murillo & Caamaño, 2013). El presente trabajo sigue esa línea con un protocolo de medición repetible, abierto y acompañado de un repositorio con el código y los datos crudos.

El objetivo fue comparar empíricamente el ordenamiento por inserción y el quicksort en C++ en términos de tiempo de ejecución, comparaciones, intercambios, profundidad de recursión, memoria auxiliar aproximada y estabilidad. Se plantearon tres hipótesis descriptivas, sin pretensión inferencial: (H1) el tiempo de inserción crecerá de forma aproximadamente cuadrática con n, salvo en entradas ya ordenadas, y el de quicksort de forma cercana a n log n; (H2) inserción superará a quicksort en entradas ascendentes y será competitiva en entradas casi ordenadas; (H3) inserción será estable y quicksort no, lo que se hará visible en entradas con claves repetidas.

2. Método

2.1 Algoritmos y variantes

Se compararon dos algoritmos internos (en memoria), uno directo y uno logarítmico. El primero es el ordenamiento por inserción en su versión canónica: recorre el arreglo y desplaza cada elemento hacia la izquierda hasta su lugar dentro del tramo ya ordenado (Cormen et al., 2022). Es iterativo, estable y usa memoria auxiliar constante. El segundo es el *quicksort* recursivo con partición de Lomuto (Cormen et al., 2022) y una única variante respecto de la versión canónica: el elemento central del tramo se intercambia con el último y este se usa como pivote, para evitar el peor caso en entradas ordenadas. La partición envía a la izquierda los elementos menores o iguales que el pivote. No se aplicó corte a inserción en tramos pequeños, mediana de tres ni eliminación de la recursión de cola, de modo que la comparación se hace entre versiones canónicas. La Tabla 1 resume su comportamiento teórico (Knuth, 1998; Cormen et al., 2022).

Tabla 1

Complejidad teórica y estabilidad de los algoritmos comparados

Algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria auxiliar	Estable
Inserción	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Sí
Quicksort (Lomuto)	O(n log n)	O(n log n)	$O(n^2)$	O(log n) esperada; O(n) peor caso (pila)	No

Nota. Fuente: elaboración propia a partir de Knuth (1998) y Cormen et al. (2022).

2.2 Entorno de ejecución

Todas las condiciones se ejecutaron en el mismo equipo y de forma secuencial (un proceso, un hilo): una laptop HP con procesador AMD Ryzen 7 7445HS a 3.20 GHz de frecuencia base (6 núcleos, 12 hilos; caché L3 de 16 MB), 8 GB de RAM (7.23 GB utilizables), almacenamiento SSD y Windows 11 Home Single Language (versión 26H2, compilación 26300.9711) de 64 bits. Las tarjetas gráficas (NVIDIA GeForce RTX 3050 y AMD Radeon 740M) no intervinieron, pues los algoritmos se ejecutan solo en CPU. El equipo permaneció conectado a la corriente durante las mediciones. Se compiló con g++ 15.2.0 (MSYS2) y las banderas -O2 y -std=c++11. Las mediciones se hicieron en una sola corrida completa, sin lanzar otros procesos pesados a propósito, y no se repitieron en horarios distintos.

2.3 Generación de datos

Se generaron arreglos de enteros con cinco patrones: aleatorio uniforme (valores entre 0 y 999 999), ascendente, descendente, casi ordenado (arreglo ascendente con el 5 % de los elementos permutados al azar) y con duplicados (15 valores únicos repetidos). Los tamaños fueron $n \in \{1\,000, 10\,000, 100\,000\}$ y cada condición (algoritmo × patrón × tamaño) se repitió 30 veces, lo que da 900 ejecuciones en total, 450 por algoritmo. Los datos se produjeron con el generador *mt19937*; la semilla de la repetición r fue $2024 + r$ (2025 a 2054) y se registró en cada fila de resultados. Ambos algoritmos recibieron en cada repetición copias idénticas de la misma entrada. No se omitió n = 100 000 en el quicksort recursivo: la profundidad máxima observada (unos 6 800 niveles en el peor patrón) no agotó la pila.

2.4 Métricas y medición

El tiempo de ejecución (ms) se midió con *std::chrono::steady_clock* alrededor de la llamada al algoritmo, sin incluir la copia de datos, la verificación del resultado ni la escritura de archivos. Además se contaron las comparaciones entre elementos, los intercambios (en inserción se cuentan los desplazamientos de elementos, pues no intercambia pares) y la profundidad máxima de recursión (máximo de llamadas anidadas, incluida la inicial; en inserción es cero). Los contadores forman parte del código medido y afectan a ambos algoritmos, por lo que los tiempos son válidos para comparar entre sí y no como tiempos absolutos de una implementación sin instrumentar. La estabilidad se evaluó con un programa independiente que ordena pares (clave, posición original) con n = 1 000 y semilla 2025, y cuenta los pares contiguos de claves iguales que quedan en orden distinto al original. La corrección de cada ordenamiento se verificó fuera del tiempo medido.

2.5 Análisis

Para cada combinación (algoritmo, patrón, n) se calculó la media, la mediana, la desviación estándar muestral, el mínimo y el máximo del tiempo. Para contrastar con la complejidad teórica se estimó la pendiente de la recta log-log del tiempo medio entre n = 1 000 y n = 100 000: una pendiente cercana a 2 indica crecimiento cuadrático y una cercana a 1.1 crecimiento tipo n log n en este rango (valor teórico: 1.11). El análisis es descriptivo, sin pruebas de hipótesis. El código, los datos crudos y las gráficas están en el repositorio público del proyecto (versión medida: commit [hash del commit final]).

3. Resultados

La Tabla 2 y la Figura 1 muestran el tiempo medio según n. En inserción el tiempo medio con n = 100 000 fue de 1 645.0 ms en entradas aleatorias, 3 335.4 ms en descendentes y 0.0858 ms en ascendentes. En quicksort fue de 4.29, 1.41 y 1.10 ms, respectivamente. Con duplicados, inserción tardó 1 573.4 ms y quicksort 220.9 ms.

Tabla 2

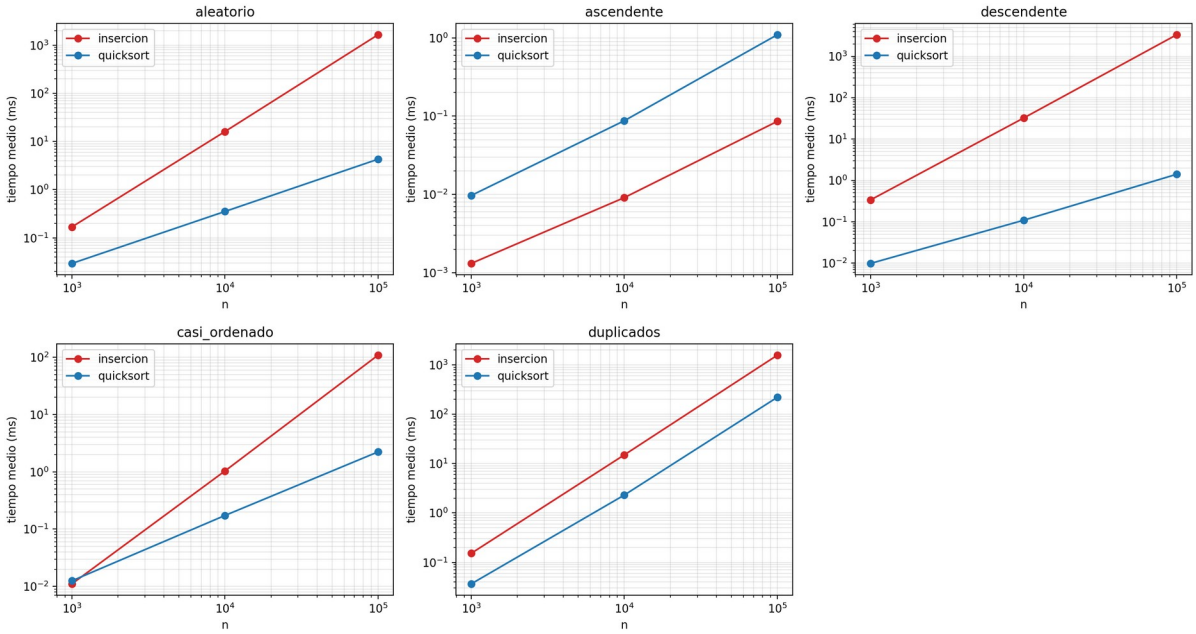
Tiempo de ejecución medio ± desviación estándar (ms) según el tamaño de entrada

Algoritmo	**Patrón**	**n = 1 000**	**n = 10 000**	**n = 100 000**
Inserción	Aleatorio	0.168 ± 0.0149	16.1 ± 0.524	1 645.0 ± 39.9
	Ascendente	0.0013 ± 0.0000	0.0091 ± 0.0016	0.0858 ± 0.0020
	Descendente	0.338 ± 0.0532	32.3 ± 0.911	3 335.4 ± 63.0
	Casi ordenado	0.0111 ± 0.0011	1.04 ± 0.0435	108.6 ± 5.95
	Duplicados	0.152 ± 0.0079	15.0 ± 0.459	1 573.4 ± 46.1
Quicksort	Aleatorio	0.0294 ± 0.0037	0.352 ± 0.0122	4.29 ± 0.0886
	Ascendente	0.0097 ± 0.0015	0.0873 ± 0.0139	1.10 ± 0.265
	Descendente	0.0098 ± 0.0079	0.109 ± 0.0073	1.41 ± 0.383
	Casi ordenado	0.0125 ± 0.0012	0.173 ± 0.0029	2.23 ± 0.327
	Duplicados	0.0366 ± 0.0021	2.30 ± 0.0297	220.9 ± 7.01

Nota. 30 repeticiones por condición. Fuente: elaboración propia (results/resumen_estadistico.csv).

Figura 1

Tiempo medio de ejecución frente al tamaño de entrada, por patrón (escala log-log).



Nota. Fuente: elaboración propia.

La Tabla 3 detalla la estadística descriptiva con n = 100 000. La variabilidad relativa fue baja en inserción (desviación estándar de entre 1.9 % y 5.5 % de la media) y mayor en quicksort, sobre todo en tiempos de pocos milisegundos (hasta 27 % en entradas descendentes y 24 % en ascendentes), como muestra la Figura 2.

Tabla 3

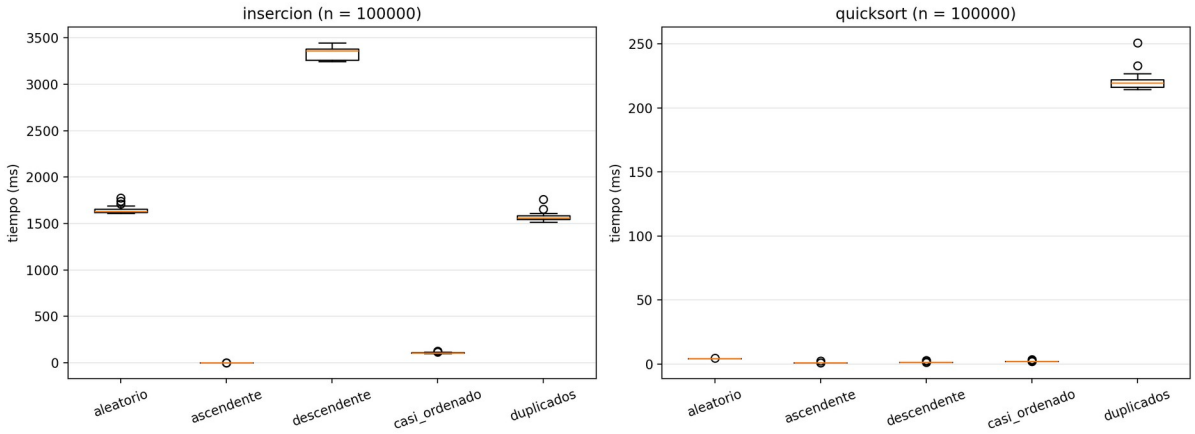
Estadística descriptiva del tiempo de ejecución (ms) con n = 100 000

Algoritmo	Patrón	Media	Mediana	Desv. est.	Mínimo	Máximo
Inserción	Aleatorio	1 645.0	1 627.8	39.9	1 610.0	1 773.2
	Ascendente	0.0858	0.0854	0.0020	0.0844	0.0957
	Descendente	3 335.4	3 359.2	63.0	3 246.0	3 446.0
	Casi ordenado	108.6	106.6	5.95	100.8	127.1
	Duplicados	1 573.4	1 561.6	46.1	1 514.2	1 759.2
Quicksort	Aleatorio	4.29	4.28	0.0886	4.18	4.68
	Ascendente	1.10	1.04	0.265	1.02	2.49
	Descendente	1.41	1.30	0.383	1.26	2.90
	Casi ordenado	2.23	2.13	0.327	2.10	3.68
	Duplicados	220.9	219.6	7.01	214.5	250.9

Nota. Fuente: elaboración propia (results/resumen_estadistico.csv).

Figura 2

Diagrama de cajas del tiempo de ejecución con n = 100 000, por patrón y algoritmo (30 repeticiones).



Nota. Fuente: elaboración propia. Las escalas verticales difieren entre paneles.

La Tabla 4 presenta las operaciones contadas. En inserción las comparaciones coincidieron con lo esperado teóricamente: 4 999.950 millones en entradas descendentes (n(n − 1)/2), 2 500.141 millones en aleatorias (≈ n²/4) y 99 999 en ascendentes (n − 1). En quicksort con datos aleatorios se hicieron 2.022 millones de comparaciones y la profundidad media fue de 40.5 niveles; con duplicados las comparaciones subieron a 333.8 millones y la profundidad a 6 807 niveles.

Tabla 4

Operaciones contadas con n = 100 000 (media de 30 repeticiones)

Algoritmo	Patrón	Comparaciones (millones)	Intercambios (millones)	Profundidad de recursión
Inserción	Aleatorio	2 500.141	2 500.041	—
	Ascendente	0.100	0.000	—
	Descendente	4 999.950	4 999.950	—
	Casi ordenado	161.134	161.034	—
	Duplicados	2 330.908	2 330.808	—
Quicksort	Aleatorio	2.022	1.153	40.5
	Ascendente	1.469	0.846	17.0
	Descendente	1.513	0.900	22.0
	Casi ordenado	2.089	1.348	60.0
	Duplicados	333.782	333.755	6 806.9

Nota. En inserción, los intercambios son desplazamientos de elementos; la profundidad no aplica (algoritmo iterativo). Fuente: elaboración propia.

Las pendientes log-log del tiempo medio entre n = 1 000 y n = 100 000 fueron, en inserción, de 2.00 a 2.01 para los patrones aleatorio, descendente, casi ordenado y duplicados, y de 0.91 para el ascendente (valor algo subestimado, pues con n = 1 000 ese tiempo es de apenas 1.3 µs, cerca de la resolución del reloj). En quicksort fueron de 1.03 a 1.13 para los cuatro primeros patrones y de 1.89 para duplicados.

En cuanto a estabilidad, con n = 1 000 y claves aleatorias (casi sin repetidos) ambos algoritmos mantuvieron el orden original de las claves iguales (0 pares alterados). Con el patrón de duplicados, inserción tuvo 0 pares alterados y quicksort 479.

4. Discusión

H1 (crecimiento cuadrático frente a n log n). Se cumplió en general. Las pendientes de inserción cercanas a 2 confirman el comportamiento cuadrático salvo en entradas ascendentes (0.91, aproximadamente lineal, con n − 1 comparaciones). Las de quicksort, entre 1.03 y 1.13, se acercan al valor teórico de n log n en este rango (1.11). La excepción fue quicksort con duplicados (1.89): con solo 15 valores distintos y la condición ≤, cada tramo de claves iguales se parte de forma totalmente desbalanceada, y el costo se vuelve cuadrático dentro de cada grupo. Esto explica que la profundidad (≈ 6 807) sea cercana a n/15 ≈ 6 667 y que las comparaciones (333.8 millones) se aproximen a $n^2/30$ ≈ 333 millones.

H2 (inserción en entradas ascendentes y casi ordenadas). Se cumplió de forma parcial. En entradas ascendentes inserción fue 13 veces más rápida que quicksort con n = 100 000, porque solo compara cada elemento con su vecino. En entradas casi ordenadas (5 % permutado) solo fue competitiva con n pequeño: con n = 1 000 ambos tardaron prácticamente lo mismo (0.0111 ms frente a 0.0125 ms), pero con n = 100 000 inserción fue 49 veces más lenta. La razón es que cada elemento permutado queda lejos de su posición y aporta del orden de n/3 inversiones, de modo que el costo sigue siendo cuadrático (pendiente 2.00), solo con una constante menor. Este resultado depende del porcentaje de desorden elegido.

H3 (estabilidad). Se cumplió. Inserción no alteró el orden relativo de ninguna clave igual, mientras que quicksort lo alteró en 479 pares con duplicados, porque los intercambios de la partición saltan elementos con la misma clave. Con enteros simples esto no cambia el arreglo final, ya que dos enteros iguales son indistinguibles; sí importa al ordenar registros por una clave y conservar un orden previo por otra, caso en el que quicksort exigiría una clave de desempate o un algoritmo estable.

Recursión y memoria. Inserción usa memoria auxiliar constante. Quicksort necesita pila: con datos aleatorios la profundidad fue de unos 40 niveles (del orden de log n), pero con duplicados llegó a más de 6 800, lo que muestra que el uso de pila en el peor caso es proporcional a n. Una variante que recurra primero sobre el tramo menor, o una partición de tres vías, acotaría ese riesgo. Aun así, con duplicados quicksort (220.9 ms) fue más rápido que inserción (1 573 ms), aunque 52 veces más lento que consigo mismo en datos aleatorios.

Limitaciones y amenazas a la validez. (a) Todos los datos provienen de un solo equipo (una laptop con Windows 11, conectada a la corriente, con procesador de frecuencia variable) y de una sola corrida, por lo que el tiempo absoluto puede variar en otro hardware o con carga distinta del sistema; las conclusiones sobre el orden de crecimiento y las razones entre algoritmos son más robustas que los milisegundos. (b) Los contadores forman parte del código medido. (c) Solo se probaron tres tamaños, de modo que las pendientes son estimaciones gruesas. (d) Los tiempos menores a unos pocos microsegundos (inserción en entradas ascendentes con n = 1 000) están cerca de la resolución del reloj. (e) Los resultados dependen de las variantes elegidas (pivote central, condición ≤, enteros de 32 bits) y del porcentaje de

desorden y de valores únicos definidos; con otro pivote o con partición de tres vías el comportamiento con duplicados sería distinto. (f) El análisis es descriptivo y no permite inferir significancia estadística.

5. Conclusiones

1. El tiempo de inserción creció de forma cuadrática en todos los patrones excepto en el ascendente, donde fue lineal; el de quicksort creció cerca de n log n, salvo con muchos valores repetidos.

2. Con 100 000 enteros aleatorios quicksort fue unas 384 veces más rápido que inserción (4.29 ms frente a 1 645 ms).

3. Inserción solo superó a quicksort en entradas ascendentes (13 veces más rápida) y se igualó a él en entradas casi ordenadas únicamente con n = 1 000.

4. Inserción es estable y quicksort no: con duplicados alteró el orden de 479 pares de claves iguales (n = 1 000).

5. Con pocos valores distintos, la partición de Lomuto con la condición ≤ degrada a quicksort hacia un costo cuadrático y una recursión profunda; conviene una partición de tres vías o un pivote aleatorio si se esperan duplicados.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, *5*(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Murillo Morera, J. de D., & Caamaño Polini, S. (2013). Comparación entre algoritmos recursivos e iterativos y su medición en términos de eficiencia. *UNICIENCIA*, *27*(1), 341–350.

Salgado Gallegos, M., Pérez Merlos, J. C., & Albarrán Trujillo, S. E. (2022). Algoritmos de ordenamiento, un análisis estadístico comparativo. *Ciencia e Ingeniería*, *43*(3), 273–280. https://erevistas.saber.ula.ve/index.php/cienciaeingenieria/article/view/18150

Sedgewick, R. (1978). Implementing Quicksort programs. *Communications of the ACM*, *21*(10), 847–857. https://doi.org/10.1145/359619.359631